

Finding 4: the folding-token ceiling is a restart ladder, and under the target-wise encoding it is never binding

M1.3 Bucket 3 (M13-C3) — evidence and reading

Working research synthesis

4 August 2026

Status. Evidence record and bounded reading. **Not** a locked Bucket 3 claim. All learner outputs cited here are pre-existing H4/H6/H7b artefacts; this document adds a trace-level audit of those artefacts (`band_replay_audit.py`) and no new learner runs.

1 The finding

(a) **Mechanism.** `folding_steps(M)` does not bound the number of folds a run performs. It bounds a *restart ladder*: nd learning starts at `tokens(1)`, and whenever the entire search at allowance T is exhausted without a solution, `genT/6` sets $T+1$ and restarts the whole search, while $T+1 \leq M$.

(b) **Saturation.** Under the target-wise exact-value encoding, every folding call terminates after *exactly one* folding step, leaving $T - 1$ tokens unspent. The allowance is therefore never binding, and a restart at a higher allowance re-executes an identical search.

(c) **Consequence.** The ladder is a strict dichotomy. If a target is solved, it is solved inside band 1 and M has no effect whatsoever. If it is not, the run performs exactly M copies of one exhausted search: cost is affine in M and the outcome cannot change.

(d) **Measured.** On the H0 roots, trace length is *exactly* affine in M with zero residual at $M \in \{1, 2, 5, 10\}$, and bands $1, \dots, M$ are byte-identical after normalising generated assumption indices, Prolog variable numbers, and the printed token counter. At the inherited default $M = 10$ this costs $10.5\times$ and $9.8\times$ the wall-clock of $M = 1$ for the two roots, for a provably identical result.

This is a statement about the learner, not about the collider fixture. Parts (a) and (b) are read off `gen.pl`, `folding.pl`, `rote_learning.pl` and the generated `bk.aba`; part (c) follows; part (d) is the H6a measurement, plus an audit over all 41 nd cells produced in this investigation.

2 What `folding_steps` actually controls

Two separate mechanisms carry the token count, and conflating them is what makes the parameter look like a fold counter.

Inner: the allowance bounds folding steps within one call. `tokens(T)` is a dynamic fact initialised to 1 in `folding.pl`. It is read once per `folding/3` call and threaded as a countdown through `fold_nd_wtc/7`, which consumes one unit per folding step:

```

% folding.pl -- one token per folding step, then DONE or FAIL
fold_nd_wtc(C,_,_,Fs,[], Fs,C) :-
  C>=0,
  write(' '), write(C), write(': DONE'), nl.
fold_nd_wtc(C,Rs,H,FsI,[T|Ts], Fs0,Co) :-
  C>=1,
  select_rule(Rs,T,R1),
  rule_hd(R1,I), rule_hd(R1,F), rule_bd(R1,[T|As]),
  write(' '), write(C), write(': folding '), show_term([T|Ts]), ...
  match(As,Ts,Asms, _,NewTs,ResTs),
  append(ResTs,NewTs, Ts1),
  C1 is C-1,
  fold_nd_wtc(C1,Rs,H,[F|FsI],Ts1, Fs0,Co).
fold_nd_wtc(C,_,_,_,[T|Ts], _,C) :-
  C==0,
  write(' '), write(C), write(': FAIL - No more folding tokens left for '), ...

```

The trace is therefore self-documenting. A call that started with allowance T and consumed k tokens prints T : folding ... down to $T-k+1$: folding, then $T-k$: DONE. A call that ran out prints 0 : FAIL - No more folding tokens left.

Outer: M bounds how many times the search is restarted. `genT/6` has two clauses. The first runs the search. The second fires only when the first has failed *completely* — that is, when every folding choice, assumption reuse and assumption introduction reachable at the current allowance has been tried and refuted:

```

% gen.pl -- the restart ladder
genT(R2,Ep0,En0,Ep,En, Ro) :-
  gen1(R2,Ep0,En0,Ep,En, Ro).
genT(R2,Ep0,En0,Ep,En, Ro) :-
  lopt(folding_mode(nd)),
  retract(tokens(T)),
  T1 is T+1,
  lopt(folding_steps(M)),
  ( T1 > M -> fail ; true ),
  nl, write('* Increasing folding tokens to: '), write(T1), nl,
  assert(tokens(T1)),
  genT(R2,Ep0,En0,Ep,En, Ro).

```

The recursive call re-enters `genT` with the *same* $R2$, E^+ and E^- . Nothing is carried over from the failed attempt except the global fresh name counter `current_new_pred/1`, which is why each band re-mints the same assumptions under higher indices. That side effect is a useful witness: it makes the restart visible in the trace.

3 Why the allowance is never binding here

The loop in Figure 1 is only useful if a larger allowance unlocks folds that a smaller one could not reach. That requires some folding call to have run out of tokens. Under the target-wise encoding, none can.

Proposition 1 (single-step folding). *In the target-wise binary exact-value encoding, every rule offered to `nd` folding has a body consisting of exactly one row-identifier equality, and every background rule has a body consisting of exactly one row-identifier equality. Consequently one folding step empties the to-be-folded list, and every `folding/3` call consumes exactly one token.*

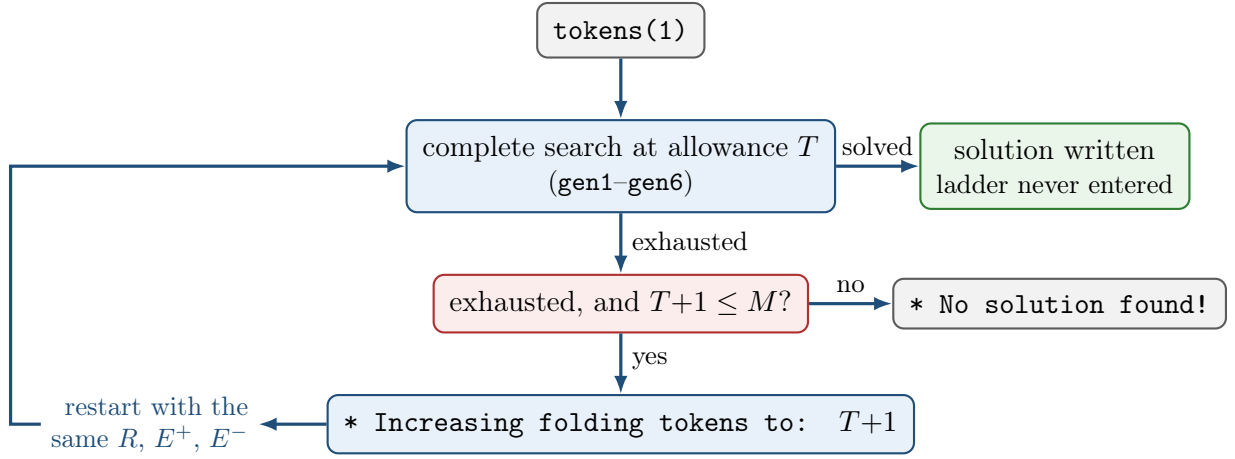


Figure 1: The two-level control. `tokens(T)` bounds folding steps inside one call; M bounds only how many times the outer loop restarts, and that loop is entered only after a *complete* failure. Everything a restart does is therefore a repetition of work already refuted, except for whatever genuinely new folds a larger allowance unlocks.

The two halves are read directly off the artefacts. Rote learning emits body-free facts, which `new_rule/3` normalises into a single equality: `e_rote_learn(E, R) :- new_rule(E, [], R)`, printed as `ert: a(A) <- [A=1]`. The generated background knowledge contains nothing else either — `input/bk.aba` for target a consists solely of rules of the form

```

b_val_1(A) :- A=1.
b_val_0(A) :- A=8.
c_val_0(A) :- A=8.

```

and contains no rule body with two atoms. So in the recursive clause of `fold_nd_wtc/7` the selected rule contributes $As = []$, hence `match([], Ts, ...)` returns $NewTs = []$ and $ResTs = Ts$, which is already `[]` for a single-equality body. The next call takes the `DONE` clause. The continuation predicate `fold_nd_aux/5` cannot re-enter folding either: its non-trivial clause needs `nselect/4`, which requires a folded list of at least two elements.

Lemma 2 (saturation implies replay). *If a complete search at allowance T fails and no folding call printed `FAIL - No more folding tokens left`, then the search at allowance $T + 1$ explores the same folding results and fails identically.*

Reason. The counter is read only by `fold_nd_wtc/7`, where it appears in the guard $C \geq 1$ of the recursive clause and the guard $C = 0$ of the failure clause. Raising the initial value only raises the depth at which those guards bite. If no branch reached the failure clause, every branch terminated through the counter-independent `DONE` clause, so the set of folding results is unchanged; the rest of `gen1-gen6` is a deterministic function of those results and of E^+, E^- . $\square$

Corollary 3. *Under Proposition 1, for every target either the search succeeds in band 1, in which case `folding_steps` is inert, or it fails and the run executes exactly M identical bands before reporting `completed_no_solution`.*

Lemma 2 is stated in terms of the printed `FAIL` diagnostic on purpose: that makes the hypothesis checkable in any run, including runs under encodings for which Proposition 1 does not hold. This is what turns the finding into a usable test rather than a fact about one fixture (Section 6).

4 Evidence

4.1 E1 — the cost is exactly affine in M , and nothing else moves

H6a varies only `folding_steps` on the frozen H0 table. The four configurations are identical apart from M , and the learner-visible `bk.aba`, `examples.json` and `data.csv` are identical per target across all four.

M	target	outcome	runtime (s)	trace lines	bands	NEW α	KO
1	a	completed_no_solution	6.638	549	1	9	16
2	a	completed_no_solution	13.117	1047	2	18	32
5	a	completed_no_solution	32.775	2541	5	45	80
10	a	completed_no_solution	69.952	5031	10	90	160
1	b	completed_no_solution	6.797	555	1	9	16
2	b	completed_no_solution	13.487	1062	2	18	32
5	b	completed_no_solution	32.825	2583	5	45	80
10	b	completed_no_solution	66.473	5118	10	90	160
1,2,5,10	c	solved	1.271–1.304	134	1	1	1

Every per-band quantity is constant, so every total is affine in M with no fitted slack:

$$\begin{aligned} \text{lines}_a(M) &= 51 + 498 M, & \text{lines}_b(M) &= 48 + 507 M, \\ \text{NEW}(M) &= 9 M, & \text{KO}(M) &= 16 M, & \text{token increases} &= M - 1. \end{aligned}$$

Both line formulae reproduce all four measurements exactly. The constants are the one-off option/rote-learning preamble (50 lines for a , 47 for b) plus the terminal `* No solution found!` line.

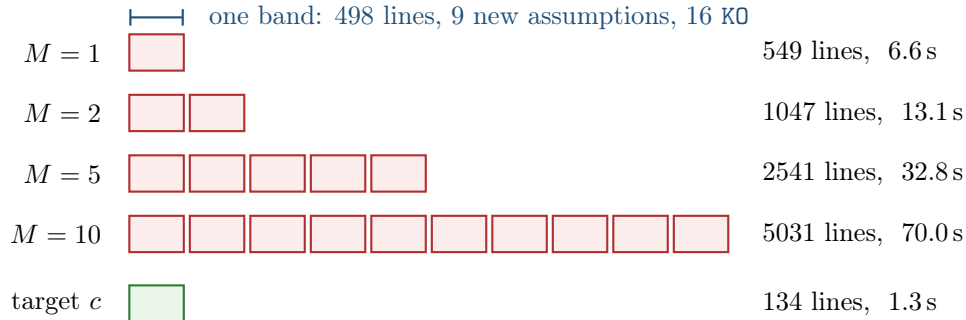


Figure 2: H6a for target a , drawn as what the trace actually contains. Each red block is one complete exhausted search, and all blocks in a row have the same normalised hash; raising M appends copies rather than extending a derivation, which is why the totals are exactly affine and every row ends in `completed_no_solution`. The solved child c (green) occupies one block at every M because it never reaches the ladder.

4.2 E2 — the bands are the same search, not merely the same size

Splitting each root trace at the `* Increasing folding tokens to:` lines and hashing each band after normalising the three things that *must* differ between restarts — generated assumption indices `alpha_N`, Prolog internal variable numbers, and the printed token counter — gives one hash for the whole ladder.

Trace (H0, $n=30$, $M=10$)	band lengths	bands	distinct hashes
baseline_cautious target a	50+498, 498×8 , 498+1	10	1
baseline_cautious target b	47+507, 507×8 , 507+1	10	1

The two extra terms are the only asymmetries: band 1 carries the one-off option and rote-learning preamble, and the last band carries the terminal `* No solution found!` line. Every band *body* hashes the same.

Before normalisation the only residual differences between two interior bands are Prolog variable numbers, e.g.

```
band 2, line 555: 2: folding [A=1] with b_val_1(_7934): b_val_1(A) <- [A=1]
band 3, line 1053: 3: folding [A=1] with b_val_1(_7894): b_val_1(A) <- [A=1]
```

and bands 3 through 9 are identical even in those. The token counters printed at each fold are exactly `folding@T` followed by `DONE@T-1` in every band, at every M , for every target: one token consumed, $T-1$ unspent. This is Proposition 1 visible in the trace, and it is the reason the bands coincide.

4.3 E3 — the solved control is untouched by the parameter

Child c solves inside band 1 at all four budgets, with a 134-line trace, a runtime of 1.27–1.30 s, and a `delta.aba` that is byte-identical across all four collections:

```
sha256 f9f75b60631b2a2aaac951521823e073bbfd5c73bd7aa96d259ca45cd426865a

c(A) :- alpha_1(A), a_val_1(A).
c_alpha_1(A) :- b_val_0(A).
assumption(alpha_1(A)).
contrary(alpha_1(A),c_alpha_1(A)) :- assumption(alpha_1(A)).
```

This is the first half of Corollary 3 realised: a successful run exits through the first `genT` clause and never reads `folding_steps` at all. It also rules out the obvious alternative explanation for E1 — that larger M inflates traces generally — since the same parameter change on the same table leaves this cell bit-for-bit unchanged.

4.4 E4 — the same ladder under a different assumption policy

The H7b arm `nd_cautious_sechk` differs from `baseline_cautious` only in `asm_intro(sechk)` instead of `relto`. It is a genuinely different search: 74 folding calls per band instead of 30, $2.7\times$ the band length and $2.4\times$ the runtime. The ladder structure is nevertheless identical.

Arm ($M=10$)	target	runtime (s)	trace lines	bands	band body
baseline_cautious (relto)	a	69.952	5031	10	498 lines, all identical
nd_cautious_sechk (sechk)	a	167.617	13731	10	1368 lines, all identical
baseline_cautious (relto)	b	66.473	5118	10	507 lines, all identical
nd_cautious_sechk (sechk)	b	172.883	13818	10	1377 lines, all identical
nd_cautious_sechk (sechk)	c	2.242	200	1	solved in band 1

So the ladder cost is not a property of one assumption-introduction policy: it multiplies whatever

the per-band search happens to cost. Under `sechk` the inherited $M = 10$ spends roughly 150s per root on provably redundant replay.

4.5 E5 — saturation holds in every nd run in the investigation

Proposition 1 is a claim about the encoding, so it should hold outside H0. `band_replay_audit.py` checks it over every target-wise `prolog.stdout` whose *effective* options include `folding_mode(nd)`: 41 cells spanning three fixtures (including the pre-pivot four-node diamond), seven nd bundles, brave and cautious semantics, `relto` and `sechk`, and $n \in \{30, 50, 60, 90\}$.

Audited property over the 41 nd cells	Count
folding calls that consumed more than one token	0
FAIL - No more folding tokens left lines	0
to-be-folded lists with more than one atom	0
solved cells that entered the ladder (bands > 1)	0 (of 27 solved)
completed_no_solution cells that entered the ladder	12 (of 14; the other 2 are $M=1$)
laddered cells whose interior bands are not identical after normalisation	0

Two rows carry most of the generality. Zero token-exhaustion failures means the hypothesis of Lemma 2 has held in every nd run ever produced here. Zero solved cells with more than one band means no solution in this investigation has ever depended on a second folding token — so on the present evidence `folding_steps(1)` would have produced every solution that `folding_steps(10)` produced, at a tenth of the cost on the failures.

5 Why the extra bands could not have helped

Corollary 3 explains why the replays are identical, but not why band 1 fails. Those are different questions, and keeping them apart is what stops Finding 4 from being read as evidence about root learnability.

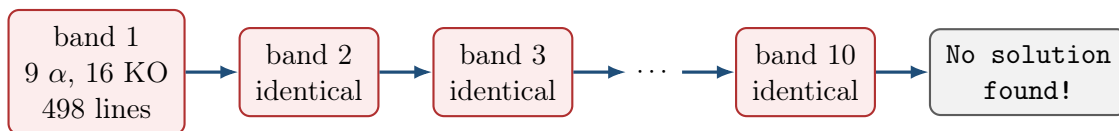
Band 1 is a *complete* refutation at allowance 1: for target a it mints 9 assumptions and records 16 K0 verdicts before exhausting its choices, including the decisive cautious refusal of the residual reuse that the brave arm accepts,

```
folding result: c_alpha_3(A) <- [b_val_0(A)]
found: alpha_2/1 ... K0, cannot introduce an assumption for [b_val_0(A)]
```

The obstruction behind that verdict lives in $\langle E^+, E^- \rangle$, not in the fold: rows 9 and 26 present identical predictor literals with opposite target labels, so no body over b, c value predicates separates them at any depth. That analysis belongs to Finding 3 and is not re-argued here. The point for Finding 4 is only this: the ladder raises fold depth, the obstruction is not fold depth, and ABALEARN has no mechanism for noticing the difference. It spends its largest search budgets precisely where extra depth is irrelevant.

6 Design implication

The useful consequence is not “pick a smaller number”. It is that the ladder already prints the information needed to decide whether continuing can possibly help, so a *sound* stopping test is available at no cost.



the obstruction is the row-9 / row-26 label conflict in $\langle E^+, E^- \rangle$, which no fold depth removes

Figure 3: At $M = 10$ the run pays ten complete refutations of the same information-level conflict. The budget parameter and the reason for failure are about different things.

Token-saturation test. On a complete failure at allowance T , continue to $T + 1$ only if some folding call exhausted its allowance. Otherwise stop: by Lemma 2 the next band is the same search.

This differs in kind from a heuristic budget or a novelty detector. It does not guess; it checks a condition under which the remaining work is provably redundant. Three consequences follow for the present codebase.

1. **For the current encoding the test always fires at $T = 1$.** By Proposition 1, an allowance of one suffices for every target-wise cell in scope. The inherited ceiling of ten — which comes from `configs/ecai2024_config.pl` and was copied unchanged into the `baseline_cautious`, `nd_brave_sechk` and `nd_cautious_sechk` configurations — multiplies the cost of every unsolved nd target by ten for no possible gain. Measured: $69.95 \rightarrow 6.64$ s and $66.47 \rightarrow 6.80$ s on the two H0 roots.
2. **The ladder is not useless in general.** It is useless exactly when the allowance is not binding. A background theory with multi-atom rule bodies would leave residual atoms to fold after the first step, the FAIL diagnostic would appear, and higher allowances would unlock genuinely new folds. So this finding constrains the *encoding-parameter pairing*, not the parameter.
3. **Search budget is the wrong lever for this failure mode.** On a target whose examples contain a predictor-label conflict, no folding budget helps. If unproductive search is to be avoided rather than merely truncated, the decision has to be made from the example set — which is where a later Causal ABA layer could contribute, by supplying independent evidence about which predictor sets are worth attempting at all. That is a design hypothesis, not a result.

7 What is not established

- **Not a Bucket 3 claim.** The canonical hub records no approved claim, and this document does not create one.
- **Not a statement about all ABA Learning.** Proposition 1 is contingent on the target-wise exact-value encoding, in which both rote rules and background rules have single-equality bodies. It says nothing about encodings with intensional or multi-atom background rules, where the allowance can bind.
- **Lemma 2 is argued from the control flow of `fold_nd_wtc/7`, not machine-checked.** Its empirical consequence — band identity — is verified on all 12 ladder cells, which is corroboration, not proof.
- **$M = 1$ sufficiency is verified, not proven, for solved cells.** All 27 solved nd cells used one band, and the four H6a *c* cells are byte-identical across M . The predicted $M = 1$ behaviour of the `nd_cautious_sechk` roots has *not* been run.
- **Runtime is affine only to within measurement noise.** Line counts are exactly affine; per-band runtime varies between 6.56 and 7.00s for target *a*, so the $M = 10$ to $M = 1$ ratio is 10.54 rather than exactly 10.
- **Root `completed_no_solution` is not mechanism recovery.** Roots *a* and *b* carry evaluator

status `no_observed_parent_deterministic_rule`; the outcome is the absence of a cautiously acceptable covering theory, and Finding 4 is about the cost of establishing that absence, not about its correctness.

- **Single mechanism family.** Every fixture cited here has a deterministic AND child. Nothing in Sections 2–3 depends on the mechanism, but the measurements do not yet demonstrate that.

8 Candidate reinforcement (not run; for discussion)

Listed for the run-planning conversation, in decreasing value per unit of compute. None of these has been executed.

1. **Falsify or confirm the band prediction on a second search bundle.** Run `nd_cautious_sechk` at `folding_steps(1)` on the H0 table. Prediction from E4: outcome unchanged, target-*a* trace = 1419 lines equal to band 1 of the $M = 10$ trace, runtime near 17s. This is the cheapest test that could refute the finding.
2. **Mirror the M sweep on a different deterministic mechanism.** A binary fixture with stochastic roots and a non-AND deterministic child would show that the affine law and band identity do not depend on the mechanism, only on the encoding.
3. **Exhibit a binding allowance.** A background theory containing one multi-atom rule should make the FAIL - No more folding tokens left diagnostic appear and make a higher allowance materially different. This would complete the dichotomy and show that the saturation test discriminates rather than always firing.

A Provenance and reproduction

Fixture and sample: `causal/fixtures/specs/m13_bucket3_binary_collider_and.yaml`; frozen samples/`n30_seed42.csv`, with

```
sha256 1edae465d0f6b6b662b3c4ed4863af21c1d9b0c1cf428774731dd3f43282a8ca
```

Learner collections, all under `causal/outputs/aba_learning/targetwise/m13_bucket3_binary_collider_and/`: `baseline_cautious_steps1`, `baseline_cautious_steps2`, `baseline_cautious_steps5`, `baseline_cautious` (the reused H4 steps-10 collection), and `nd_cautious_sechk`, each at `n30_seed42`.

The H6a collections were produced by (H6 record, not re-run for this document):

```
python -m causal.targetwise.cli run \  
  --config causal/configs/targetwise/m13_bucket3_binary_collider_and/  
  baseline_cautious_steps1/n30_seed42.yaml  
python -m causal.targetwise.cli run \  
  --config causal/configs/targetwise/m13_bucket3_binary_collider_and/  
  baseline_cautious_steps2/n30_seed42.yaml  
python -m causal.targetwise.cli run \  
  --config causal/configs/targetwise/m13_bucket3_binary_collider_and/  
  baseline_cautious_steps5/n30_seed42.yaml
```

The trace audit reported in E2 and E5 is reproduced from the repository root by:

```
python docs/experiments/qualitative/M13-C3-binary-collider-and/band_replay_audit.py
```

Engine files consulted read-only: `folding.pl` (tokens/1, fold_nd_wtc/7, fold_nd_aux/5), `gen.pl` (genT/6, gen_new_name/1), `rote_learning.pl` (e_rote_learn/2), `asp_utils.pl` (new_rule/3).

Configurations: `configs/baseline_cautious_config.pl` and its `steps1/steps2/steps5` variants, `configs/nd_cautious_sechk_config.pl`, `configs/ecai2024_config.pl`.

Primary experiment records: `h6_folding_and_n_ablation.md / .tex` (H6a/H6b), `h4_cautious_vs_brave.md` (the $M = 10$ root cost being explained), `h7b_cautious_relto_vs_sechk.md` (the sechk arm), `experiment.md` (investigation hub).

Line-count convention. “Trace lines” counts lines of text in `output/prolog.stdout`, matching the figures tabulated in the H6 record. Root traces have no terminating newline on their final line, so `wc -l` reports one fewer for those cells; the audit script normalises this.